

Group 1

By Question number

Q1)

The Child process must instantly print its own Process ID and its parent's Process ID using the format: [Child] My PID is X, and my Parent's PID is Y.

```
printf("[Child] My PID is %d, and my Parent's PID is %d.\n", getpid(), getppid());
```

Q2)

The child process must then simulate a slow data-transmission delay by suspending its execution for exactly 4 seconds. After waking up, it must print [Child] Data transmission finished. Exiting.

```
sleep(4);
```

Q3)

The Parent process must immediately print its own Process ID using the format: [Parent] Monitoring Active. PID: X.

```
printf(" [Parent] Monitoring. Active. PID: %d.\n", getpid());
```

Q4)

The parent must not run ahead; it must explicitly block and pause execution until the child process finishes its execution to prevent creating a Zombie process.

```
printf(" [Parent] Monitoring. Active. PID: %d.\n", getpid());
```

```
wait(NULL);
```

Q5)

Once the parent detects the child has exited, it should print [Parent] Child process reaped successfully. Application exiting.

```
printf(" [Parent] Child process reaped successfully. Application exiting.\n");
```

Complete Program

```
#include<stdio.h>

#include<unistd.h>

#include<sys/wait.h>

int main() {

pid_t pid = fork();

if (pid == 0) {
```

```

printf("[Child] My PID is %d, and my Parent's PID is %d.\n", getpid(), getppid());

sleep(4);

printf(" [Child] Data transmission finished. Exiting.\n");

}

else {

printf(" [Parent] Monitoring. Active. PID: %d.\n", getpid());

wait(NULL);

printf(" [Parent] Child process reaped successfully. Application exiting.\n");

}

return 0;

}

```

```

GNU nano 8.7.1 group1.c
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("[Child] My PID is %d, and my Parent's PID is %d.\n", getpid(), getppid());

        sleep(4);

        printf(" [Child] Data transmission finished. Exiting.\n");
    }
    else {
        printf(" [Parent] Monitoring. Active. PID: %d.\n", getpid());

        wait(NULL);

        printf(" [Parent] Child process reaped successfully. Application exiting.\n");
    }
    return 0;
}

```

[Read 23 lines]

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-B Previous
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-G Copy ^E Where Was M-F Next

Compilation

```

└─[kaveesh@fedora] - [~/Documents/University/University notes/2nd Year/4th semester/0bsidian/FIS
semester 4/OS/practical] - [Tue Jun 09, 15:37]
└─[$] <> gcc group1.c -o group_1

```

Output

```
[kaveesh@fedora] - [~/Documents/University/University notes/2nd Year/4th semester/Obsidian/FIS smester 4/OS/practical] - [Tue Jun 09, 15:37]  
[$] <> ./group_1  
[Parent] Monitoring. Active. PID: 11660.  
[Child] My PID is 11661, and my Parent's PID is 11660.  
[Child] Data transmission finished. Exiting.  
[Parent] Child process reaped successfully. Application exiting.
```

```
./group_1  
[Parent] Monitoring. Active. PID: 11660.  
[Child] My PID is 11661, and my Parent's PID is 11660.  
[Child] Data transmission finished. Exiting.  
[Parent] Child process reaped successfully. Application exiting.
```